§11.4.170 Host-Render Visual Proof — ota-manager LoginPage

§11.4.170 Host-Render Visual Proof — ota-manager LoginPage (first increment)

Revision: 2 **Last modified:** 2026-07-09T16:56:00Z **Scope:** `clients/ota-manager` frontend only (harness added under `clients/ota-manager/visual/`). **Mandate:** §11.4.170 — device-independent HOST-side rendered-pixel visual proof, per screen×state×{light,dark}, dual-validated by (i) golden image-diff AND (ii) an OCR/vision layout oracle.

---

1. What was stood up

The FIRST provable increment of a §11.4.170 host-render harness for `ota-manager`: the **real `LoginPage` component** (`src/features/auth/login-page.tsx`, imported — not copied) is rendered to real PNG files ON THE HOST by headless Chromium via Playwright, with **no dev server, no emulator, no running app**. The component is mounted with the exact providers it depends on (`QueryClientProvider` + TanStack-Router memory history — `LoginPage → useLogin → useMutation/useNavigate`).

Harness stack (installed, versions captured):

Tool	Version	Role
Node	v24.18.0	runtime
Playwright (core)	1.55.1	host render → PNG + real rendered bounding boxes

Tool	Version	Role
Chromium (playwright build)	v1193 (140.0.7339.186)	rendering engine — `~/.cache/ms-playwright/chromium-1193`
Vite	6.4.3	builds the harness to a static bundle (`visual/.out`)
@vitejs/plugin-react + @tailwindcss/vite	(project devDeps)	real React 19 + Tailwind v4 pipeline
tailwindcss	4.3.1	real token/utility CSS from `src/index.css`
pixelmatch	6.0.0	oracle (i) golden image-diff
pngjs	7.0.0	PNG decode/encode
tesseract	5.3.0 (system binary)	oracle (ii) OCR text-reading

Install evidence: `pnpm install` + `pnpm add -D playwright@1.55.1 pixelmatch@6.0.0 pngjs@7.0.0` both succeeded; `pnpm exec playwright install chromium` downloaded the matching browser (the cached chromium-1228 did not match the pinned 1.55.1, which needs chromium-1193 — honest version reconciliation, no fake). Chromium launch was verified before use.

2. Screens × states rendered

Representative screen: **LoginPage** (title "OTA Manager", description, Email + Password fields, "Sign in" button) — in BOTH themes, driven exactly by the `src/index.css` token contract:

- **light** → `document.documentElement.classList = "light"` (light HSL token overrides)
- **dark** → `document.documentElement.classList = "dark"` (tokens fall back to `:root` = dark palette)

Baseline (golden) PNGs, committed:

- `baselines/login-light.png` — 1000×720, real light-theme render
- `baselines/login-dark.png` — 1000×720, real dark-theme render

Both were visually confirmed as genuine, correctly-themed, distinct renders of the real login screen (not blank).

3. Dual-oracle result (both themes PASS)

From `results.json` / `run.log` (re-run deterministic, exit 0):

theme	(i) image-diff golden-good	(i) image-diff golden-bad	(ii) OCR labels	(ii) layout baseline	(ii) layout golden-bad
light	0.0000% → **PASS** (matches baseline)	0.7136% → **FLAGGED**	ALL PRESENT	OK	FLAGGED collapsed submit (398.0×0.0)
dark	0.0000% → **PASS** (matches baseline)	1.4653% → **FLAGGED**	ALL PRESENT	OK	FLAGGED collapsed submit (398.0×0.0)

Oracle (i) — golden image-diff (`oracle-diff.mjs`, pixelmatch): re-render vs committed baseline differs by 0.0000% (identical → no false positive). A deliberately-mutated render (golden-bad: the submit button collapsed to height 0 — the §11.4.170 forensic "broken button while token tests stay green" case) differs by 0.71%/1.47% and is correctly **FLAGGED**. Diff visualizations written to `diff/login-{light,dark}-{good,bad}-diff.png`.

Oracle (ii) — OCR + rendered-bounds layout (`oracle-ocr.mjs`): Tesseract read all expected labels ("OTA Manager", "Email", "Password", "Sign in") from BOTH baseline PNGs (`ocr/login-{light,dark}.txt`). The layout oracle over the REAL Playwright bounding boxes confirmed no element is collapsed / clipped / off-screen / overlapping on the baseline, and correctly **FLAGGED** the collapsed submit button on the golden-bad render (`bounds/login-{theme}-{good,bad}.json`).

4. Analyzer self-validation (§11.4.107(10) golden-good + golden-bad)

Neither oracle can silently bluff — each was proven to PASS its golden-good input AND FAIL/flag its golden-bad input:

- `image_diff_analyzer_sound: true` (good=0.0000% pass, bad≥0.2% flagged, both themes)
- `layout_analyzer_sound: true` (baseline clean, golden-bad collapse detected, both themes)
- `ocr_analyzer_sound: true` (baseline reads all labels, golden-bad flags the suppressed label, both themes — see §4a for the M1 closure detail)

This is the concrete rebuttal to a value/token-equality test being the sole proof (§11.4.170 forbids that): the collapsed-button regression is invisible to a hex/sp/dp equality unit test but is caught by BOTH rendered-pixel oracles here.

4a. OCR oracle self-validation (M1 closure)

Gap closed. The prior increment (Revision 1) self-validated the image-diff and layout oracles with golden-good + golden-bad fixtures, but the **OCR oracle (`oracle-ocr.mjs`, Tesseract 5.3.0) ran only against the unmutated baseline** — it was never shown to FAIL on a missing/garbled label. `results.json` therefore carried `image_diff_analyzer_sound` + `layout_analyzer_sound` but **no** `ocr_analyzer_sound`. Per §11.4.107(10) an analyzer that is never exercised against a golden-bad input is an unproven (potential rubber-stamp) gate. This section closes that Minor finding (M1) with captured evidence.

The OCR mutation. A new producer `blankTextRegion(srcPng, outPng, rect)` (`oracle-ocr.mjs`) paints a flat opaque rectangle over the **real Playwright rendered bounds** of one on-screen label — the title **"OTA Manager"** (bounds key `title`) — inflated by 4 px to also cover glyph anti-alias fringes, destroying the glyphs so the label physically cannot be OCR-read. The exact rectangle painted is recorded in `results.json` (`ocr.golden_bad.blank_rect`, e.g. light = `{x:297,y:222,width:406,height:40}`). The golden-bad PNGs are committed at `mutated/login-{light,dark}-ocrbad.png` and their re-OCR

output at `ocr/login-{light,dark}-ocrbad.txt`. This mutation is orthogonal to the image-diff and layout golden-bads (a collapsed submit button) — it targets rendered *text* specifically.

Result (both themes, deterministic across two consecutive runs):

theme	OCR golden-good (baseline)	OCR golden-bad (title painted over)
light	ALL PRESENT (`OTA Manager`, `Email`, `Password`, `Sign in`) → **PASS**	missing = `["OTA Manager"]`, other 3 still read → **FLAGGED**
dark	ALL PRESENT → **PASS**	missing = `["OTA Manager"]`, other 3 still read → **FLAGGED**

- **Golden-good PASS** — `ocrText(baseline)` reads all four expected labels (`ocr/login-{light,dark}.txt`); `ocrLabelsPresent(...).ok = true`.
- **Golden-bad FLAGGED** — `ocrText(ocrbad.png)` on the painted-over render no longer reads the title; `ocrLabelsPresent` reports `missing: ["OTA Manager"]`, `ok:false`, and the runner sets `ocr.golden_bad.detected = true`. The other three labels remain present, proving the flag is specific to the suppressed text and not a blanket failure. (The description line "…access the OTA management dashboard." is *not* a false-positive match for `"OTA Manager"` — "ota management" ≠ "ota manager".)
- **Negative control** — the golden-good run is itself the rubber-stamp guard: on the un-blanked image the oracle does **not** report `OTA Manager` missing (`detected` would be `false`), so `ocr_analyzer_sound` can only be `true` when the mutation genuinely removed the text AND the oracle noticed.

Soundness gate. `run-all.mjs` now computes `ocr_analyzer_sound = THEMES.every(t => ocr.ok && ocr.golden_bad.detected)` and folds it into the hard-fail condition alongside the other two analyzers. `results.json` `self_validation` now reads:

```
image_diff_analyzer_sound: true
layout_analyzer_sound:     true
ocr_analyzer_sound:        true      ← NEW (M1 closure)
```

Exact command output (`pnpm hostrender`, exit 0, run twice with byte-identical verdicts):

```
[light]
  ocr golden-good        : ALL PRESENT
  ocr golden-bad         : FLAGGED missing "OTA Manager"
(missing=["OTA Manager"])
[dark]
  ocr golden-good        : ALL PRESENT
  ocr golden-bad         : FLAGGED missing "OTA Manager"
(missing=["OTA Manager"])
---- analyzer self-validation ----
  image-diff analyzer sound : true
  layout   analyzer sound   : true
  ocr      analyzer sound   : true
OVERALL: PASS
```

The image-diff and layout self-validations are unchanged and still PASS — this increment only **adds** the OCR self-validation.

5. Reproduction

From `clients/ota-manager/`:

```
pnpm install
pnpm add -D playwright@1.55.1 pixelmatch@6.0.0 pngjs@7.0.0   #
first time only
pnpm exec playwright install chromium                        #
first time only
pnpm hostrender          # build harness + render + dual-oracle
self-validation
```

Exit 0 = all gates + both analyzer self-validations PASS. Full log: `run.log`.

6. Evidence file manifest (this directory)

- `results.json` — structured machine-readable verdicts (source of truth)
- `run.log` — full `pnpm hostrender` console output (exit 0)
- `baselines/login-{light,dark}.png` — committed golden baselines
- `rerender/login-{light,dark}.png` — identical re-render (golden-good input)
- `mutated/login-{light,dark}-bad.png` — mutated render (image-diff/layout golden-bad input)
- `mutated/login-{light,dark}-ocrbad.png` — title painted over (OCR golden-bad input, M1)

- `diff/login-{light,dark}-{good,bad}-diff.png` — pixelmatch diff visualizations
- `bounds/login-{light,dark}-{good,bad}.json` — real rendered bounding boxes
- `ocr/login-{light,dark}.txt` — Tesseract OCR output per baseline (OCR golden-good)
- `ocr/login-{light,dark}-ocrbad.txt` — Tesseract OCR of the painted-over render (OCR golden-bad, M1)
- `harness-src/*` — copy of the harness source for reproducibility

Harness source of record lives in-tree at `clients/ota-manager/visual/` (`harness.{html,tsx,css}`, `vite.harness.config.ts`, `lib-render.mjs`, `oracle-diff.mjs`, `oracle-ocr.mjs`, `run-all.mjs`) + `pnpm hostrender` script.

7. Honest findings & scope boundaries (§11.4.6)

- **Theming-wiring gap (real finding, FACT — not fixed here):** the `ui-store` holds a `theme` value and `topbar.tsx` toggles it, but **no code applies a `.dark`/`.light` class to `document.documentElement`/`body`** (verified: the only match across `src/` is the `@custom-variant dark` declaration in `index.css`). At runtime the toggle changes only the Sun/Moon icon, not the actual palette — `:root` (dark) is always in effect. The harness therefore drives the theme the way `index.css` *defines* it (light=`.light`, dark=`:root`), proving both token sets render correctly; wiring the toggle to a DOM class is a separate app fix (out of scope for this first increment).
- This increment covers ONE screen in its default (empty-form) state. Error/validation/ loading states and other screens (dashboard, devices, releases, …) are follow-on §11.4.170 coverage, not claimed here.
- The `data-theme` attribute the artifact/theme-toggle convention uses elsewhere is not part of this app; the app's actual contract is the `.light`/`:root` class split above, which is what was reproduced. No behaviour is claimed that was not rendered and captured.